

System Binary Proxy Execution: Electron Applications

Other sub-techniques of System Binary Proxy Execution (14)

Adversaries may abuse components of the Electron framework to execute malicious code. The Electron framework hosts many common applications such as Signal, Slack, and Microsoft Teams. ^[1] Originally developed by GitHub, Electron is a cross-platform desktop application development framework that employs web technologies like JavaScript, HTML, and CSS. ^[2] The Chromium engine is used to display web content and Node.js runs the backend code. ^[3]

Due to the functional mechanics of Electron (such as allowing apps to run arbitrary commands), adversaries may also be able to perform malicious functions in the background potentially disguised as legitimate tools within the framework. ^[3] For example, the abuse of `teams.exe` and `chrome.exe` may allow adversaries to execute malicious commands as child processes of the legitimate application (e.g., `chrome.exe --disable-gpu-sandbox --gpu-launcher="C:\Windows\system32\cmd.exe /c calc.exe"`). ^[4]

Adversaries may also execute malicious content by planting malicious [JavaScript](#) within Electron applications. ^[5]

ID: T1218.015

Sub-technique of: [T1218](#)

ⓘ

Tactic: [Stealth](#)

ⓘ

Platforms: Linux, Windows, macOS

Contributors: Debabrata Sharma; Uriel Kosayev

Version: 2.0

Created: 07 March 2024

Last Modified: 12 May 2026

[Version Permalink](#)

Procedure Examples

ID	Name	Description
C0057	3CX Supply Chain Attack	During the 3CX Supply Chain Attack , AppleJeus leveraged the 3CX application's electron framework to execute its malicious libraries under the official 3CX electron application. ^[6]
S1213	Lumma Stealer	Lumma Stealer as leveraged Electron Applications to disable GPU sandboxing to avoid detection by security software. ^[7]

Mitigations

ID	Mitigation	Description
M1042	Disable or Remove Feature or Program	Remove or deny access to unnecessary and potentially vulnerable software and features to prevent abuse by adversaries. Many native binaries may not be necessary within a given environment: for example, consider disabling the Node.js integration in all renderers that display remote content to protect users by limiting adversaries' power to plant malicious JavaScript within Electron applications. ^[8]
		Where possible, enforce binary and application integrity with digital signature verification to prevent untrusted code from executing. For example, do not use <code>shell.openExternal</code> with untrusted content.
M1038	Execution Prevention	Where possible, set <code>nodeIntegration</code> to false, which disables access to the Node.js function. ^[9] By disabling access to the Node.js function, this may limit the ability to execute malicious commands by injecting JavaScript code.
		Do not disable <code>webSecurity</code> , which may allow for users of the application to invoke malicious content from online sources.
M1050	Exploit Protection	Microsoft's Enhanced Mitigation Experience Toolkit (EMET) Attack Surface Reduction (ASR) feature can be used to block methods of using trusted binaries to bypass application control.
		Ensure that Electron is updated to the latest version and critical vulnerabilities (such as <code>nodeIntegration</code> bypasses) are patched and cannot be exploited.

Detection Strategy

ID	Name	Analytic ID	Analytic Description
DET0025	Detecting Electron Application Abuse for Proxy Execution	AN0071	Abuse of trusted Electron apps (Teams, Slack, Chrome) to spawn child processes or execute payloads via malicious command-line arguments (e.g., <code>--gpu-launcher</code>) and modified app resources (.asar). Behavior chain: suspicious parent process (Electron app) → unusual command-line args → child process creation → optional DLL/network artifacts.
		AN0072	Abuse of Linux Electron binaries by modifying app.asar or config JS files and spawning unexpected child processes (bash, curl, python).
		AN0073	Abuse of macOS Electron apps by modifying app.asar bundles and spawning child processes (osascript, curl, sh) from Electron executables.

References

1. [Trend Micro. \(2023, June 6\). Abusing Electronbased applications in targeted attacks. Retrieved March 7, 2024.](#)
2. [Alanna Titterington. \(2023, September 14\). Security of Electron-based desktop applications. Retrieved March 7, 2024.](#)
3. [TOM ABAI. \(2023, August 10\). There's a New Stealer Variant in Town, and It's Using Electron to Stay Fully Undetected. Retrieved March 7, 2024.](#)
4. [Kosayev, U. \(2023, June 15\). One Electron to Rule Them All. Retrieved March 7, 2024.](#)
5. [ElectronJS.org. \(n.d.\). Retrieved March 7, 2024.](#)
6. [Robert Falcone, Josh Grunzweig. \(2023, March 30\). Threat Brief: 3CXDesktopApp Supply Chain Attack. Retrieved September 15, 2025.](#)
7. [Buddy Tancio, Fe Cureg, and Jovit Samaniego, Trend Micro. \(2025, January 30\). Lumma Stealer's GitHub-Based Delivery Explored via Managed Detection and Response. Retrieved March 22, 2025.](#)
8. [Stack Overflow. \(n.d.\). Why do I see an "Electron Security Warning" after updating my Electron project to the latest version?. Retrieved March 7, 2024.](#)
9. [CertiK. \(2020, June 30\). Vulnerability in Electron-based Application: Unintentionally Giving Malicious Code Room to Run. Retrieved March 7, 2024.](#)